PES University

Department of Computer Science and Engineering

Database Management Systems: Experiential Learning, Level 3 Project

ShopSense

A Hybrid Semantic Product Discovery Engine with Association-Rule Recommendations

PostgreSQL 17 + pgvector + Next.js

SRN	Name
PES_____	Akshat Sharma
PES_____	Teammate name

Course instructor: Dr. Nagasundari S

https://github.com/Akshatpy/DBMS_MiniProject

1. Title

ShopSense: A Hybrid Semantic Product Discovery Engine with Association-Rule Recommendations

DBMS Experiential Learning, Level 3 Project · Database option 2 (SQL + Vector database) · Web interface in Next.js.

Source code (public): https://github.com/Akshatpy/DBMS_MiniProject

2. Students (SRN and Name)

#	SRN	Name
1	PES_____	Akshat Sharma
2	PES_____	Teammate name

3. Abstract

Most online shops still search by exact keywords. That works when a shopper already knows the product name, and it fails when they describe a need: a search for *"something to keep my dog's teeth clean"* returns nothing useful because the catalogue says "dental chews". Searching by meaning alone has the opposite weakness and gets exact model names wrong. ShopSense is a multi-role marketplace (customer, seller, admin) whose database layer handles both.

The system keeps its structured data (users, sellers, catalogue, carts, orders, payments) in **PostgreSQL 17**, and its unstructured text (product descriptions and customer reviews) as 384-dimensional embeddings in the same database using **pgvector**, with HNSW indexes and cosine distance. A single SQL function runs a **hybrid search**: it fuses full-text ranking, product-vector similarity and review-vector similarity with Reciprocal Rank Fusion, and applies ordinary filters (category, price, rating, stock) in the same query. A trigger-fed queue and a background worker keep embeddings in step with edits.

Beyond search, the project demonstrates the database concepts of the course: a 3NF schema with domains and constraints, atomic and concurrency-safe checkout using row locks, 15 triggers, views and materialized views, window functions and `ROLLUP` for analytics, association rules (support, confidence, lift) for "frequently bought together", and role-based access enforced inside the database with Row-Level Security. Search quality was measured on 58 labelled queries, including a held-out set. Vector search raised nDCG@10 from about 0.4–0.5 to 0.66–0.74 over keyword search, and the final hybrid design matches or slightly exceeds semantic-only search while keeping exact-match behaviour. The first hybrid design was actually worse than semantic-only search; the report notes how that was found and fixed.

4. Tech Stack

Layer	Technology	Used for
Relational database	PostgreSQL 17.11	schema, constraints, triggers, transactions, views, Row-Level Security
Vector database	pgvector 0.8.7 (inside PostgreSQL)	`vector(384)` columns, HNSW indexes, cosine distance search
Other PostgreSQL features	`pgcrypto`, `pg_trgm`, full-text search (`tsvector`, GIN)	bcrypt passwords, typo-tolerant suggestions, keyword search
Embedding model	all-MiniLM-L6-v2 via transformers.js (ONNX, CPU)	text to 384-d vectors, run locally with no external API
Web framework	Next.js 15 (App Router), React 19, TypeScript	pages, server components, route handlers
Styling and charts	Tailwind CSS 4, Recharts	interface and dashboards
Auth	`jose` (signed JWT in an HTTP-only cookie), bcrypt inside the database	sessions and password hashing
Validation	Zod (request shape) plus database constraints (values)	input validation
DB driver	`pg`	connection pool, transactions
Background worker	Node.js, `LISTEN/NOTIFY`, `FOR UPDATE SKIP LOCKED`	embedding queue
Container	Docker (`pgvector/pgvector:pg17`)	reproducible database
Testing	Node test runner (22 database integration tests)	constraints, triggers, RLS, concurrency
Version control	Git and GitHub	source history
Data	Amazon Reviews 2023 (McAuley Lab, UCSD): 3,001 products, 14,116 reviews; synthetic users and orders	catalogue and review text

Database scale after seeding: 800 customers, 25 sellers, 15,417 orders (29,492 order lines), 17,117 stored vectors.

5. SQL Database Commands

Every excerpt below is taken directly from the repository; file names and line numbers are shown above each block. The schema is applied in order by `npm run db:setup` (files `db/01` to `db/07`).

5.1 DDL: extensions, types and validation domains

db/01_extensions_types.sql (line 7)

```
CREATE EXTENSION IF NOT EXISTS vector;
CREATE EXTENSION IF NOT EXISTS pgcrypto;
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- ---------- Enumerated types (closed sets of values) ----------
CREATE TYPE user_role      AS ENUM ('customer', 'seller', 'admin');
CREATE TYPE order_status   AS ENUM ('pending', 'paid', 'shipped', 'delivered', 'cancelled');
CREATE TYPE payment_method AS ENUM ('upi', 'card', 'netbanking', 'cod');
CREATE TYPE payment_status AS ENUM ('pending', 'success', 'failed', 'refunded');
CREATE TYPE search_mode    AS ENUM ('lexical', 'semantic', 'hybrid');

-- ---------- Domains: reusable, database-level input validation ----------
CREATE DOMAIN email_address AS TEXT
  CHECK (VALUE ~* '^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$');

CREATE DOMAIN money_inr AS NUMERIC(12, 2)
  CHECK (VALUE >= 0);

CREATE DOMAIN indian_pincode AS CHAR(6)
  CHECK (VALUE ~ '^[1-9][0-9]{5}$');

CREATE DOMAIN indian_phone AS TEXT
  CHECK (VALUE ~ '^[6-9][0-9]{9}$');

-- GSTIN: 2-digit state code, 10-char PAN, entity number, 'Z', checksum
CREATE DOMAIN gstin_code AS TEXT
  CHECK (VALUE ~ '^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z][1-9A-Z]Z[0-9A-Z]$');
```

Domains make a validation rule reusable: any column declared `indian_pincode` or `email_address` rejects bad values without repeating the rule.

5.2 DDL: tables with primary keys, foreign keys and constraints

db/02_schema.sql (line 8)

```
CREATE TABLE users (
  user_id        BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  email          email_address NOT NULL,
  password_hash  TEXT          NOT NULL,             -- bcrypt hash (pgcrypto), never plaintext
  full_name      TEXT          NOT NULL CHECK (length(btrim(full_name)) BETWEEN 2 AND 100),
  phone          indian_phone,
  role           user_role     NOT NULL DEFAULT 'customer',
  is_active      BOOLEAN       NOT NULL DEFAULT TRUE,
  created_at     TIMESTAMPTZ   NOT NULL DEFAULT now(),
```

```
  last_login_at  TIMESTAMPTZ,
  CONSTRAINT uq_users_email UNIQUE (email),
  -- target for the composite FK that guarantees every seller row belongs to a 'seller' user
  CONSTRAINT uq_users_id_role UNIQUE (user_id, role),
  CONSTRAINT chk_users_hash_is_bcrypt CHECK (password_hash LIKE '$2%')
```

db/02_schema.sql (line 54)

```
CREATE TABLE sellers (
  seller_id      BIGINT     PRIMARY KEY,
  role           user_role  NOT NULL DEFAULT 'seller' CHECK (role = 'seller'),
  store_name     TEXT       NOT NULL UNIQUE CHECK (length(btrim(store_name)) >= 2),
  gstin          gstin_code UNIQUE,
  support_email  email_address,
  is_verified    BOOLEAN    NOT NULL DEFAULT FALSE,
  joined_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  -- Composite FK: a seller row can only exist for a user whose role is 'seller',
  -- and that user's role cannot be changed while the seller row exists.
  CONSTRAINT fk_sellers_user_is_seller FOREIGN KEY (seller_id, role)
    REFERENCES users (user_id, role) ON DELETE CASCADE ON UPDATE RESTRICT
);

-- ---------------------------------------------------------------------
-- Catalog
```

The composite foreign key means a row in `sellers` can only exist for a user whose role is `seller`.

db/02_schema.sql (line 164)

```
CREATE TABLE orders (
  order_id      BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  user_id       BIGINT       NOT NULL REFERENCES users (user_id) ON DELETE RESTRICT,
  address_id    BIGINT       NOT NULL REFERENCES addresses (address_id) ON DELETE RESTRICT,
  status        order_status NOT NULL DEFAULT 'pending',
  subtotal      money_inr    NOT NULL,
  shipping_fee  money_inr    NOT NULL DEFAULT 0,
  total_amount  money_inr    NOT NULL,
  placed_at     TIMESTAMPTZ  NOT NULL DEFAULT now(),
  updated_at    TIMESTAMPTZ  NOT NULL DEFAULT now(),
  CONSTRAINT chk_orders_total CHECK (total_amount = subtotal + shipping_fee)
);

CREATE TABLE order_items (
  order_id    BIGINT    NOT NULL REFERENCES orders (order_id) ON DELETE CASCADE,
  product_id  BIGINT    NOT NULL REFERENCES products (product_id) ON DELETE RESTRICT,
  quantity    INT       NOT NULL CHECK (quantity > 0),
  unit_price  money_inr NOT NULL CHECK (unit_price > 0),       -- price snapshot at purchase time
  line_total  NUMERIC(12, 2) GENERATED ALWAYS AS (quantity * unit_price) STORED,
  PRIMARY KEY (order_id, product_id)
);

CREATE TABLE payments (
  payment_id       BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  order_id         BIGINT         NOT NULL UNIQUE REFERENCES orders (order_id) ON DELETE CASCADE,
  method           payment_method NOT NULL,
  amount           money_inr      NOT NULL,
  status           payment_status NOT NULL DEFAULT 'pending',
```

5.3 DDL: indexes

db/03_indexes.sql (line 18)

```
-- Partial indexes
CREATE INDEX idx_products_active_price ON products (category_id, price) WHERE is_active;
-- at most one default address per user
CREATE UNIQUE INDEX uq_addresses_one_default ON addresses (user_id) WHERE is_default;
-- at most one OPEN stock alert per product
CREATE UNIQUE INDEX uq_stock_alerts_open ON stock_alerts (product_id) WHERE resolved_at IS NULL;
-- at most one PENDING embedding job per entity (lets triggers use ON CONFLICT DO NOTHING)
CREATE UNIQUE INDEX uq_embedding_jobs_pending ON embedding_jobs (entity_type, entity_id) WHERE
processed_at IS NULL;
CREATE INDEX idx_embedding_jobs_queue ON embedding_jobs (enqueued_at) WHERE processed_at IS NULL;

-- Case-insensitive email lookups (login)
CREATE UNIQUE INDEX uq_users_email_lower ON users (lower(email));

-- Full-text search: GIN over the generated tsvector
CREATE INDEX idx_products_search_tsv ON products USING GIN (search_tsv);
-- Trigram index for typo-tolerant autocomplete on titles
CREATE INDEX idx_products_title_trgm ON products USING GIN (title gin_trgm_ops);
```

5.4 DDL: ALTER and DROP

`db/demo/ddl_demo.sql` walks through schema evolution on a scratch table and is safe to re-run.

db/demo/ddl_demo.sql (line 19)

```
\echo '--- 2. ALTER TABLE: add a column, backfill, then make it NOT NULL'
ALTER TABLE demo_coupons ADD COLUMN expires_on DATE;
UPDATE demo_coupons SET expires_on = current_date + 30;
ALTER TABLE demo_coupons ALTER COLUMN expires_on SET NOT NULL;

\echo '--- 3. ALTER TABLE: add a CHECK across columns, widen a type, rename a column'
ALTER TABLE demo_coupons ADD CONSTRAINT chk_demo_future CHECK (expires_on >= DATE '2024-01-01');
ALTER TABLE demo_coupons ALTER COLUMN percent_off TYPE NUMERIC(5, 2);
ALTER TABLE demo_coupons RENAME COLUMN percent_off TO discount_percent;

\echo '--- 4. constraints reject bad data (each statement below is EXPECTED to fail)'
```

db/demo/ddl_demo.sql (line 41)

```
\echo '--- 6. ALTER TABLE ... DROP CONSTRAINT / DROP COLUMN, then DROP TABLE'
ALTER TABLE demo_coupons DROP CONSTRAINT chk_demo_future;
ALTER TABLE demo_coupons DROP COLUMN category_id;
\d demo_coupons
DROP TABLE demo_coupons;
\echo '--- done: scratch table dropped'
```

`npm run db:setup -- --reset` runs `DROP SCHEMA public CASCADE` and rebuilds everything. Other `ALTER` statements in the setup files: `ALTER TABLE … ENABLE ROW LEVEL SECURITY` (13 tables) and `ALTER ROLE`

`shopsense_app PASSWORD …`.

5.5 DML: INSERT, UPDATE, DELETE, SELECT

INSERT with an upsert (add to cart; quantity is added if the product is already there):

src/app/api/cart/route.ts (line 14)

```
// Add to cart (increments if already present). Stock & quantity limits are enforced by
// the trg_cart_items_validate trigger and the quantity CHECK constraint.
export const POST = handler(async (req: Request) => {
  const s = await requireRole('customer');
  const { productId, quantity } = Item.parse(await req.json());
  await withUser(s, (c) => c.query(
    `INSERT INTO cart_items (cart_id, product_id, quantity)
     VALUES (my_cart_id(), $1, $2)
     ON CONFLICT (cart_id, product_id) DO UPDATE SET quantity = cart_items.quantity +
EXCLUDED.quantity`,
    [productId, quantity]));
  return NextResponse.json({ ok: true });
});
```

UPDATE (seller edits a product; Row-Level Security hides other sellers' rows from this statement):

src/app/api/seller/products/[id]/route.ts (line 23)

```
  const sets = entries.map(([col], i) => `${col} = $${i + 2}`).join(', ');
  const result = await withUser(s, (c) => c.query(
    `UPDATE products SET ${sets} WHERE product_id = $1 RETURNING product_id, stock_qty, updated_at`,
    [id, ...entries.map(([, v]) => v)]));
  if (!result.rowCount) throw new HttpError(404, 'Product not found or not owned by you');
  return NextResponse.json(result.rows[0]);
});
```

DELETE:

src/app/api/cart/route.ts (line 35)

```
export const DELETE = handler(async (req: Request) => {
  const s = await requireRole('customer');
  const productId = Number(new URL(req.url).searchParams.get('productId'));
  // RLS scopes this DELETE to the caller's own cart
  await withUser(s, (c) => c.query('DELETE FROM cart_items WHERE product_id = $1', [productId]));
  return NextResponse.json({ ok: true });
});
```

SELECT with JOINs, `GROUP BY`, aggregation and a correlated subquery:

src/lib/data/shop.ts (line 21)

```
export async function getOrders(session: Session) {
  return withUser(session, async (c) => {
```

```
    const { rows } = await c.query(
      `SELECT o.order_id, o.status, o.subtotal, o.shipping_fee, o.total_amount, o.placed_at,
              pay.method AS payment_method, pay.status AS payment_status,
              a.city, a.pincode,
              json_agg(json_build_object(
                'product_id', oi.product_id, 'title', p.title, 'image_url', p.image_url,
                'quantity', oi.quantity, 'unit_price', oi.unit_price, 'line_total', oi.line_total)
                ORDER BY oi.product_id) AS items,
              (SELECT json_agg(json_build_object('status', h.new_status, 'at', h.changed_at) ORDER BY
h.changed_at)
                 FROM order_status_history h WHERE h.order_id = o.order_id) AS timeline
         FROM orders o
         JOIN order_items oi USING (order_id)
         JOIN products p     ON p.product_id = oi.product_id
         JOIN addresses a    ON a.address_id = o.address_id
         LEFT JOIN payments pay ON pay.order_id = o.order_id
        WHERE o.user_id = $1
        GROUP BY o.order_id, pay.method, pay.status, a.city, a.pincode
        ORDER BY o.placed_at DESC
        LIMIT 50`, [session.uid]);
    return rows;
  });
}
```

5.6 Analytical queries: window functions, HAVING, ROLLUP

Month-over-month growth and moving average (window functions `LAG` and `AVG OVER`):

src/lib/data/seller.ts (line 21)

```
    // Monthly trend with month-over-month growth (LAG) and a 3-month moving average
    const monthly = c.query(`
      WITH m AS (
        SELECT date_trunc('month', sale_date)::DATE AS month,
               sum(line_total) AS revenue, sum(quantity) AS units, count(DISTINCT order_id) AS orders
          FROM v_sales_lines WHERE status <> 'cancelled'
         GROUP BY 1)
      SELECT month, revenue, units, orders,
             round(100 * (revenue - lag(revenue) OVER w) / NULLIF(lag(revenue) OVER w, 0), 1) AS
mom_growth_pct,
             round(avg(revenue) OVER (w ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 0)          AS
moving_avg_3m
        FROM m
      WINDOW w AS (ORDER BY month)
       ORDER BY month`);
```

Association rules for "frequently bought together": self-join, `GROUP BY … HAVING`, support, confidence and lift, stored in a materialized view:

db/06_views.sql (line 122)

```
CREATE MATERIALIZED VIEW mv_product_pairs AS
WITH baskets AS (
  SELECT oi.order_id, oi.product_id
    FROM order_items oi
    JOIN orders o USING (order_id)
```

```
   WHERE o.status <> 'cancelled'
),
n AS (SELECT count(DISTINCT order_id)::NUMERIC AS total FROM baskets),
item AS (SELECT product_id, count(*)::NUMERIC AS cnt FROM baskets GROUP BY product_id),
pairs AS (
  SELECT a.product_id AS a_id, b.product_id AS b_id, count(*)::NUMERIC AS cnt
    FROM baskets a
    JOIN baskets b ON a.order_id = b.order_id AND a.product_id <> b.product_id
   GROUP BY a.product_id, b.product_id
  HAVING count(*) >= 3                       -- minimum support count
)
SELECT pr.a_id                                          AS product_id,
       pr.b_id                                          AS related_product_id,
       pr.cnt::INT                                      AS together_count,
       round(pr.cnt / n.total, 6)                       AS support,
       round(pr.cnt / ia.cnt, 4)                        AS confidence,
       round((pr.cnt / ia.cnt) / (ib.cnt / n.total), 3) AS lift
  FROM pairs pr
  JOIN item ia ON ia.product_id = pr.a_id
  JOIN item ib ON ib.product_id = pr.b_id
 CROSS JOIN n;

CREATE UNIQUE INDEX uq_mv_product_pairs ON mv_product_pairs (product_id, related_product_id);

CREATE FUNCTION frequently_bought_together(p_product_id BIGINT, p_limit INT DEFAULT 6)
```

Revenue by category with subtotals and a grand total (`GROUP BY ROLLUP`):

db/06_views.sql (line 225)

```
CREATE FUNCTION admin_category_rollup(p_since DATE DEFAULT NULL)
RETURNS TABLE (top_category TEXT, sub_category TEXT, revenue NUMERIC, units BIGINT,
               level TEXT, share_pct NUMERIC)
LANGUAGE plpgsql STABLE SECURITY DEFINER SET search_path = public AS $$
#variable_conflict use_column
BEGIN
  PERFORM assert_admin();
  RETURN QUERY
  SELECT coalesce(top.name, c.name)                          AS top_category,
         CASE WHEN top.category_id IS NULL THEN NULL ELSE c.name END AS sub_category,
         sum(d.revenue), sum(d.units)::BIGINT,
         CASE GROUPING(coalesce(top.name, c.name), CASE WHEN top.category_id IS NULL THEN NULL ELSE
c.name END)
              WHEN 0 THEN 'subcategory' WHEN 1 THEN 'category subtotal' ELSE 'grand total' END,
         round(100 * sum(d.revenue) / (SELECT sum(x.revenue) FROM mv_daily_sales x
                                       WHERE p_since IS NULL OR x.sale_date >= p_since), 2)
    FROM mv_daily_sales d
    JOIN categories c        ON c.category_id = d.category_id
    LEFT JOIN categories top ON top.category_id = c.parent_id
   WHERE p_since IS NULL OR d.sale_date >= p_since
   GROUP BY ROLLUP (coalesce(top.name, c.name), CASE WHEN top.category_id IS NULL THEN NULL ELSE
c.name END)
   ORDER BY 1 NULLS LAST, 3 DESC;
END $$;

-- Seller leaderboard: overall rank, rank within their main category, share of platform revenue
CREATE FUNCTION admin_top_sellers(p_months INT DEFAULT 6)
RETURNS TABLE (seller_id BIGINT, store_name TEXT, main_category TEXT, revenue NUMERIC, units BIGINT,
```

5.7 Views

db/06_views.sql (line 57)

```
CREATE VIEW v_sales_lines WITH (security_invoker = true) AS
SELECT o.order_id,
       o.placed_at,
       (o.placed_at AT TIME ZONE 'Asia/Kolkata')::DATE AS sale_date,
       o.status,
       o.user_id                                       AS customer_id,
       oi.product_id,
       p.title                                         AS product_title,
       p.seller_id,
       p.category_id,
       oi.quantity,
       oi.unit_price,
       oi.line_total
  FROM order_items oi
  JOIN orders o   ON o.order_id = oi.order_id
  JOIN products p ON p.product_id = oi.product_id;

-- Open low-stock alerts with product context
```

`security_invoker = true` makes the view run with the caller's rights, so Row-Level Security applies through it.

5.8 Stored functions and procedures

Authentication runs inside the database (bcrypt verification with `pgcrypto`):

db/04_functions.sql (line 107)

```
CREATE FUNCTION authenticate(p_email TEXT, p_password TEXT)
RETURNS TABLE (user_id BIGINT, full_name TEXT, role user_role)
LANGUAGE sql SECURITY DEFINER SET search_path = public AS $$
  UPDATE users u
     SET last_login_at = now()
   WHERE lower(u.email) = lower(btrim(p_email))
     AND u.is_active
     AND u.password_hash = crypt(p_password, u.password_hash)
  RETURNING u.user_id, u.full_name, u.role
```

A stored procedure, called with `CALL`:

db/04_functions.sql (line 268)

```
CREATE PROCEDURE restock_product(p_product_id BIGINT, p_quantity INT)
LANGUAGE plpgsql AS $$
BEGIN
  IF p_quantity IS NULL OR p_quantity <= 0 OR p_quantity > 10000 THEN
    RAISE EXCEPTION 'Restock quantity must be between 1 and 10000' USING ERRCODE = '22023';
  END IF;
  UPDATE products SET stock_qty = stock_qty + p_quantity WHERE product_id = p_product_id;
  IF NOT FOUND THEN
```

```
    RAISE EXCEPTION 'Product % not found or not owned by you', p_product_id USING ERRCODE = '42501';
  END IF;
END $$;
```

The `checkout()` function (atomic order placement with row locking) is shown in the code screenshots in Section 8.

5.9 Triggers

Illegal order-status changes are rejected by a `BEFORE UPDATE` trigger:

db/05_triggers.sql (line 185)

```
CREATE FUNCTION trg_fn_order_status_guard() RETURNS TRIGGER
LANGUAGE plpgsql AS $$
BEGIN
  IF NEW.status = OLD.status THEN
    RETURN NEW;
  END IF;
  IF NOT (
       (OLD.status = 'pending' AND NEW.status IN ('paid', 'cancelled'))
    OR (OLD.status = 'paid'    AND NEW.status IN ('shipped', 'cancelled'))
    OR (OLD.status = 'shipped' AND NEW.status = 'delivered')
  ) THEN
    RAISE EXCEPTION 'Illegal order status change % -> % (order %)', OLD.status, NEW.status,
OLD.order_id
      USING ERRCODE = 'SS004';
  END IF;
  RETURN NEW;
END $$;

CREATE TRIGGER trg_orders_status_guard
  BEFORE UPDATE OF status ON orders
  FOR EACH ROW EXECUTE FUNCTION trg_fn_order_status_guard();

-- T8. Audit trail of every status change
CREATE FUNCTION trg_fn_order_status_history() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
```

Rating averages are recomputed by a statement-level trigger using a transition table, so a bulk insert of reviews recomputes each product once rather than once per row:

db/05_triggers.sql (line 166)

```
CREATE TRIGGER trg_reviews_stats_ins AFTER INSERT ON reviews
  REFERENCING NEW TABLE AS new_rows
  FOR EACH STATEMENT EXECUTE FUNCTION trg_fn_review_stats_ins();

CREATE TRIGGER trg_reviews_stats_upd AFTER UPDATE ON reviews
  REFERENCING OLD TABLE AS old_rows NEW TABLE AS new_rows
  FOR EACH STATEMENT EXECUTE FUNCTION trg_fn_review_stats_upd();

CREATE TRIGGER trg_reviews_stats_del AFTER DELETE ON reviews
  REFERENCING OLD TABLE AS old_rows
```

The low-stock alert trigger is shown in the code screenshots in Section 8.

5.10 Transactions and concurrency

Checkout runs as one transaction. Product rows are locked with `SELECT … FOR UPDATE` in primary-key order, so two buyers cannot both take the last unit and two checkouts cannot deadlock. The concurrency lab in the application runs a naive read-then-write (two units sold from a stock of one) against `checkout()` (second buyer rejected with `SS001`); the automated test `concurrency: two buyers racing for the last unit` asserts the correct outcome.

5.11 Security: roles, grants and Row-Level Security

db/07_security.sql (line 23)

```
-- Column-level privilege: the app can never read password hashes
GRANT SELECT (user_id, email, full_name, phone, role, is_active, created_at, last_login_at)
  ON users TO shopsense_app;
```

db/07_security.sql (line 85)

```
-- users: see yourself; admins see and (de)activate everyone
CREATE POLICY users_select ON users FOR SELECT
  USING (user_id = (SELECT app_user_id()) OR (SELECT is_admin()));
CREATE POLICY users_admin_update ON users FOR UPDATE
  USING ((SELECT is_admin())) WITH CHECK ((SELECT is_admin()));

CREATE POLICY profiles_owner ON customer_profiles FOR ALL
  USING (user_id = (SELECT app_user_id()) OR (SELECT is_admin()))
  WITH CHECK (user_id = (SELECT app_user_id()));
```

The web application connects as `shopsense_app`, which is not a superuser, cannot bypass Row-Level Security and cannot read `password_hash`. Each request runs in a transaction that first sets `app.user_id`; the role is looked up from the `users` table, never trusted from the client.

6. NoSQL / Vector Database Commands

The project uses **Option 2: SQL + Vector database**, with pgvector inside PostgreSQL. No separate NoSQL store was added, because no part of the data is schema-less or graph-shaped enough to justify a second system to keep consistent. The few semi-structured fields (search filters, evaluation details) use PostgreSQL `jsonb`.

Rubric question	Answer
What data becomes embeddings?	Product text (title, brand, category path, feature bullets, description) and review text (title and body)
Which model?	all-MiniLM-L6-v2, 384 dimensions, mean-pooled and L2-normalised, run locally with transformers.js; input capped at the model's 256-token training length
How are vectors stored?	`vector(384)` columns in `product_embeddings` and `review_embeddings`; a per-customer taste vector in `customer_profiles`
Similarity metric?	Cosine distance (`<=>`), similarity = 1 − distance
How is relevance retrieved?	Hybrid Reciprocal Rank Fusion of keyword, product-vector and review-vector rankings, with relational filters

6.1 Vector storage

db/02_schema.sql (line 118)

```
CREATE TABLE product_embeddings (
  product_id   BIGINT PRIMARY KEY REFERENCES products (product_id) ON DELETE CASCADE,
  embedding    vector(384) NOT NULL,
  model        TEXT        NOT NULL,
  source_hash  TEXT        NOT NULL,      -- md5 of the embedded text; lets the worker skip no-op
updates
  embedded_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE review_embeddings (
  review_id    BIGINT PRIMARY KEY REFERENCES reviews (review_id) ON DELETE CASCADE,
  embedding    vector(384) NOT NULL,
  model        TEXT        NOT NULL,
  source_hash  TEXT        NOT NULL,
  embedded_at  TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

6.2 Vector indexes

db/03_indexes.sql (line 36)

```
-- Vector similarity: HNSW graph indexes using cosine distance (<=>)
-- m = max connections per node, ef_construction = candidate list size at build time
CREATE INDEX idx_product_embeddings_hnsw ON product_embeddings
```

```
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
CREATE INDEX idx_review_embeddings_hnsw ON review_embeddings
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
```

HNSW is an approximate nearest-neighbour graph index. `m` is the number of links per node and `ef_construction` the candidate list size while building. At query time `hnsw.ef_search = 200` is set to favour recall.

6.3 What text is embedded

db/04_functions.sql (line 283)

```
CREATE FUNCTION product_embedding_text(p_product_id BIGINT) RETURNS TEXT
LANGUAGE sql STABLE AS $$
  SELECT concat_ws('. ',
           p.title,
           'Brand: ' || p.brand,
           'Category: ' || category_path(p.category_id),
           left(replace(p.features, E'\n', '; '), 700),
           left(p.description, 700))
    FROM products p
   WHERE p.product_id = p_product_id
$$;

CREATE FUNCTION review_embedding_text(p_review_id BIGINT) RETURNS TEXT
```

6.4 Keeping vectors up to date: queue trigger and worker

db/05_triggers.sql (line 69)

```
CREATE FUNCTION trg_fn_enqueue_embedding() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
  v_type TEXT := TG_ARGV[0];
  -- generic over both tables: read "<type>_id" from the row as JSON
  v_id   BIGINT := (to_jsonb(NEW) ->> (TG_ARGV[0] || '_id'))::BIGINT;
BEGIN
  INSERT INTO embedding_jobs (entity_type, entity_id)
  VALUES (v_type, v_id)
  ON CONFLICT (entity_type, entity_id) WHERE processed_at IS NULL DO NOTHING;
  PERFORM pg_notify('embedding_jobs', v_type || ':' || v_id);
  RETURN NULL;
END $$;
```

The worker claims jobs with `FOR UPDATE SKIP LOCKED`, embeds in batches, skips text whose hash is unchanged, and upserts the vector:

scripts/embed-worker.mjs (line 51)

```
        const table = d.type === 'product' ? 'product_embeddings' : 'review_embeddings';
        const key = d.type === 'product' ? 'product_id' : 'review_id';
```

```
        await client.query(
          `INSERT INTO ${table} (${key}, embedding, model, source_hash, embedded_at)
           VALUES ($1, $2::vector, $3, $4, now())
           ON CONFLICT (${key}) DO UPDATE
             SET embedding = EXCLUDED.embedding, model = EXCLUDED.model,
                 source_hash = EXCLUDED.source_hash, embedded_at = now()`,
          [d.id, toPgVector(vectors[i]), EMBEDDING_MODEL, md5(d.text)]);
```

6.5 Hybrid search by vector similarity

Query embedding in the application, then the SQL function call:

src/lib/data/search.ts (line 15)

```
  // lexical mode never needs the vector, so skip the model entirely
  const embedding = p.mode === 'lexical' ? null : toPgVector(await embedOne(q));
  const embedMs = performance.now() - t0;

  return withUser(session, async (c) => {
    const t1 = performance.now();
    const { rows } = await c.query<SearchHit>(
      `SELECT s.score, s.lexical_rank, s.semantic_rank, s.review_rank, s.semantic_similarity,
              s.matched_review_id, s.matched_review_similarity,
              r.body AS matched_review_body, r.rating AS matched_review_rating,
```

The fusion step of `search_products()` (Reciprocal Rank Fusion; the full function is in the code screenshots):

db/04_functions.sql (line 416)

```
  fused AS (
    SELECT coalesce(l.product_id, s.product_id, rv.product_id) AS product_id,
           coalesce((SELECT weight FROM lex_q) / (60 + l.rnk), 0)
         + coalesce(1.0 / (60 + s.rnk), 0)
         + coalesce(0.5 / (60 + rv.rnk), 0)                    AS score,   -- reviews are noisier:
half weight
           l.rnk  AS lexical_rank,
           s.rnk  AS semantic_rank,
           rv.rnk AS review_rank,
           1 - s.dist  AS semantic_similarity,
           rv.review_id AS matched_review_id,
           1 - rv.dist AS matched_review_similarity
      FROM lexical l
      FULL JOIN semantic s       ON s.product_id = l.product_id
      FULL JOIN review_ranked rv ON rv.product_id = coalesce(l.product_id, s.product_id)
  )
  SELECT product_id, score::DOUBLE PRECISION, lexical_rank, semantic_rank, review_rank,
         semantic_similarity, matched_review_id, matched_review_similarity
    FROM fused
```

6.6 Similar products and personalised recommendations

Nearest neighbours of a product (`LATERAL` join over the HNSW index):

db/04_functions.sql (line 439)

```
CREATE FUNCTION similar_products(p_product_id BIGINT, p_limit INT DEFAULT 8)
RETURNS TABLE (product_id BIGINT, similarity DOUBLE PRECISION)
LANGUAGE sql STABLE AS $$
  SELECT n.product_id, 1 - n.dist
    FROM product_embeddings src
   CROSS JOIN LATERAL (
     SELECT pe.product_id, pe.embedding <=> src.embedding AS dist
       FROM product_embeddings pe
       JOIN products p ON p.product_id = pe.product_id AND p.is_active
      WHERE pe.product_id <> src.product_id
      ORDER BY pe.embedding <=> src.embedding
      LIMIT p_limit
   ) n
   WHERE src.product_id = p_product_id
   ORDER BY n.dist
$$;
```

Nearest neighbours of the customer's taste vector (the mean embedding of what they bought), excluding items already bought and out-of-stock items:

db/04_functions.sql (line 458)

```
CREATE FUNCTION recommend_for_me(p_limit INT DEFAULT 12)
RETURNS TABLE (product_id BIGINT, similarity DOUBLE PRECISION)
LANGUAGE sql STABLE SECURITY DEFINER SET search_path = public AS $$
  SELECT n.product_id, 1 - n.dist
    FROM customer_profiles cp
   CROSS JOIN LATERAL (
     SELECT pe.product_id, pe.embedding <=> cp.taste_embedding AS dist
       FROM product_embeddings pe
       JOIN products p ON p.product_id = pe.product_id
      WHERE p.is_active AND p.stock_qty > 0
        AND NOT EXISTS (SELECT 1
                          FROM orders o JOIN order_items oi USING (order_id)
                         WHERE o.user_id = cp.user_id AND oi.product_id = pe.product_id)
      ORDER BY pe.embedding <=> cp.taste_embedding
      LIMIT p_limit
   ) n
   WHERE cp.user_id = app_user_id() AND cp.taste_embedding IS NOT NULL
   ORDER BY n.dist
$$;
```

6.7 Measured search quality

Labelled queries are in `eval/queries.json` (40) and `eval/holdout.json` (18, written before the fusion algorithm was revised and never used to tune it). Metrics are at k = 10; relevance is graded (2 = concept in the title, 1 = in the description or features).

Main set, 40 queries	nDCG@10	Precision@10	MRR	nDCG, natural-language	nDCG, exact
Keyword (full-text)	0.492	0.472	0.813	0.338	0.852

Main set, 40 queries	nDCG@10	Precision@10	MRR	nDCG, natural-language	nDCG, exact
Semantic (vectors)	0.660	0.677	0.855	0.575	0.857
Hybrid (RRF)	**0.687**	**0.710**	**0.880**	**0.600**	**0.890**

Held-out set, 18 queries	nDCG@10	Precision@10	MRR
Keyword	0.402	0.372	0.715
Semantic	0.733	0.733	0.889
Hybrid	**0.738**	**0.744**	0.861

The first hybrid design gave keyword and semantic rankings equal weight and scored *worse than semantic alone* (0.649 and 0.589). Weak keyword matches on natural-language queries out-voted good semantic ones. The fix uses a conjunctive keyword match when one exists and gives the OR-fallback a quarter of the weight. It improved the held-out set as well, so it is not just tuned to the queries we had inspected. Hybrid's lead over semantic on the held-out set is within noise, so we claim "at least as good as semantic, far better than keyword".

Index performance (`EXPLAIN ANALYZE`): with 85,585 vectors an exact scan takes about 80–90 ms and HNSW 4–10 ms at recall 0.95–1.00. At a few thousand vectors the difference is within noise. A GIN index speeds full-text search by 30–110 times, and the composite B-tree on orders by 24 times.

7. Next.js Project Tree Structure

Top level of the repository:

```
./
├── db/
│   ├── demo/
│   ├── 01_extensions_types.sql
│   ├── 02_schema.sql
│   ├── 03_indexes.sql
│   ├── 04_functions.sql
│   ├── 05_triggers.sql
│   ├── 06_views.sql
│   └── 07_security.sql
├── docs/
│   ├── screenshots/   (24 images)
│   ├── benchmark-results.md
│   ├── data-dictionary.md
│   ├── ER-diagram.md
│   ├── important-queries.md
│   ├── PROJECT_REPORT.md
│   ├── schema-reference.md
│   ├── ShopSense-Project-Report.pdf
│   ├── team.json
│   └── vector-database-design.md
├── eval/
│   ├── holdout.json
│   └── queries.json
├── scripts/
│   ├── benchmark.mjs
│   ├── build-pdf.mjs
│   ├── code-shots.mjs
│   ├── db-setup.mjs
│   ├── embed-worker.mjs
│   ├── evaluate.mjs
│   ├── fetch-dataset.mjs
│   ├── gen-docs.mjs
│   ├── race-demo.mjs
│   ├── screenshots.mjs
│   └── seed.mjs
├── src/
│   ├── app/
│   ├── components/
│   ├── lib/
│   └── middleware.ts
├── tests/
│   └── db.test.mjs
├── .env.example
├── docker-compose.yml
├── next.config.mjs
├── package.json
├── postcss.config.mjs
├── PROPOSAL.md
├── README.md
└── tsconfig.json
```

Application source (`src/`):

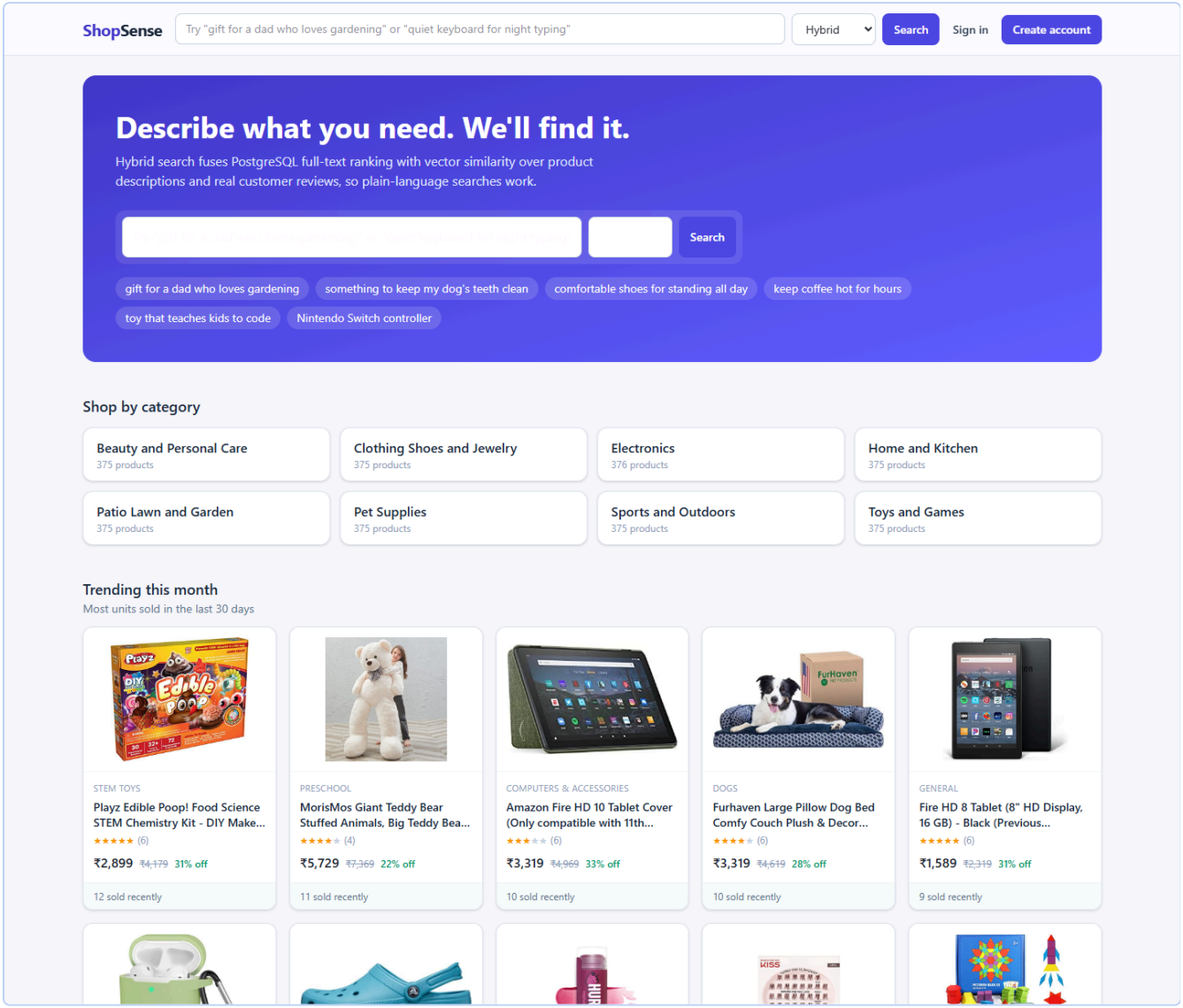
```
src/
├── app/
│   ├── admin/
│   │   ├── lab/
│   │   │   └── page.tsx
│   │   ├── search-quality/
│   │   │   └── page.tsx
│   │   ├── users/
│   │   │   ├── page.tsx
│   │   │   └── ToggleActive.tsx
│   │   ├── page.tsx
│   │   └── RefreshButton.tsx
│   ├── api/
│   │   ├── admin/
│   │   │   ├── explain/
│   │   │   │   └── route.ts
│   │   │   ├── lab/
│   │   │   │   └── route.ts
│   │   │   ├── refresh/
│   │   │   │   └── route.ts
│   │   │   └── users/
│   │   │       └── [id]/
│   │   ├── auth/
│   │   │   ├── login/
│   │   │   │   └── route.ts
│   │   │   ├── logout/
│   │   │   │   └── route.ts
│   │   │   └── register/
│   │   │       └── route.ts
│   │   ├── cart/
│   │   │   └── route.ts
│   │   ├── checkout/
│   │   │   └── route.ts
│   │   ├── orders/
│   │   │   └── [id]/
│   │   │       └── cancel/
│   │   ├── reviews/
│   │   │   └── route.ts
│   │   ├── search/
│   │   │   └── route.ts
│   │   ├── seller/
│   │   │   ├── orders/
│   │   │   │   └── [id]/
│   │   │   └── products/
│   │   │       ├── [id]/
│   │   │       └── route.ts
│   │   └── suggest/
│   │       └── route.ts
│   ├── cart/
│   │   ├── CartView.tsx
│   │   └── page.tsx
│   ├── login/
│   │   └── page.tsx
│   ├── orders/
│   │   ├── CancelButton.tsx
│   │   └── page.tsx
│   ├── product/
│   │   └── [id]/
│   │       └── page.tsx
│   ├── register/
│   │   └── page.tsx
│   ├── search/
```

```
│   │   └── page.tsx
│   ├── seller/
│   │   ├── products/
│   │   │   ├── [id]/
│   │   │   │   └── page.tsx
│   │   │   ├── new/
│   │   │   │   └── page.tsx
│   │   │   ├── page.tsx
│   │   │   └── ProductForm.tsx
│   │   ├── page.tsx
│   │   └── SellerActions.tsx
│   ├── globals.css
│   ├── layout.tsx
│   └── page.tsx
├── components/
│   ├── Charts.tsx
│   ├── Kpi.tsx
│   ├── LogoutButton.tsx
│   ├── ProductActions.tsx
│   ├── ProductCard.tsx
│   └── SearchBar.tsx
├── lib/
│   ├── auth.ts
│   ├── db.ts
│   ├── embedding.mjs
│   ├── format.ts
│   ├── session.ts
│   └── validation.ts
└── middleware.ts
```

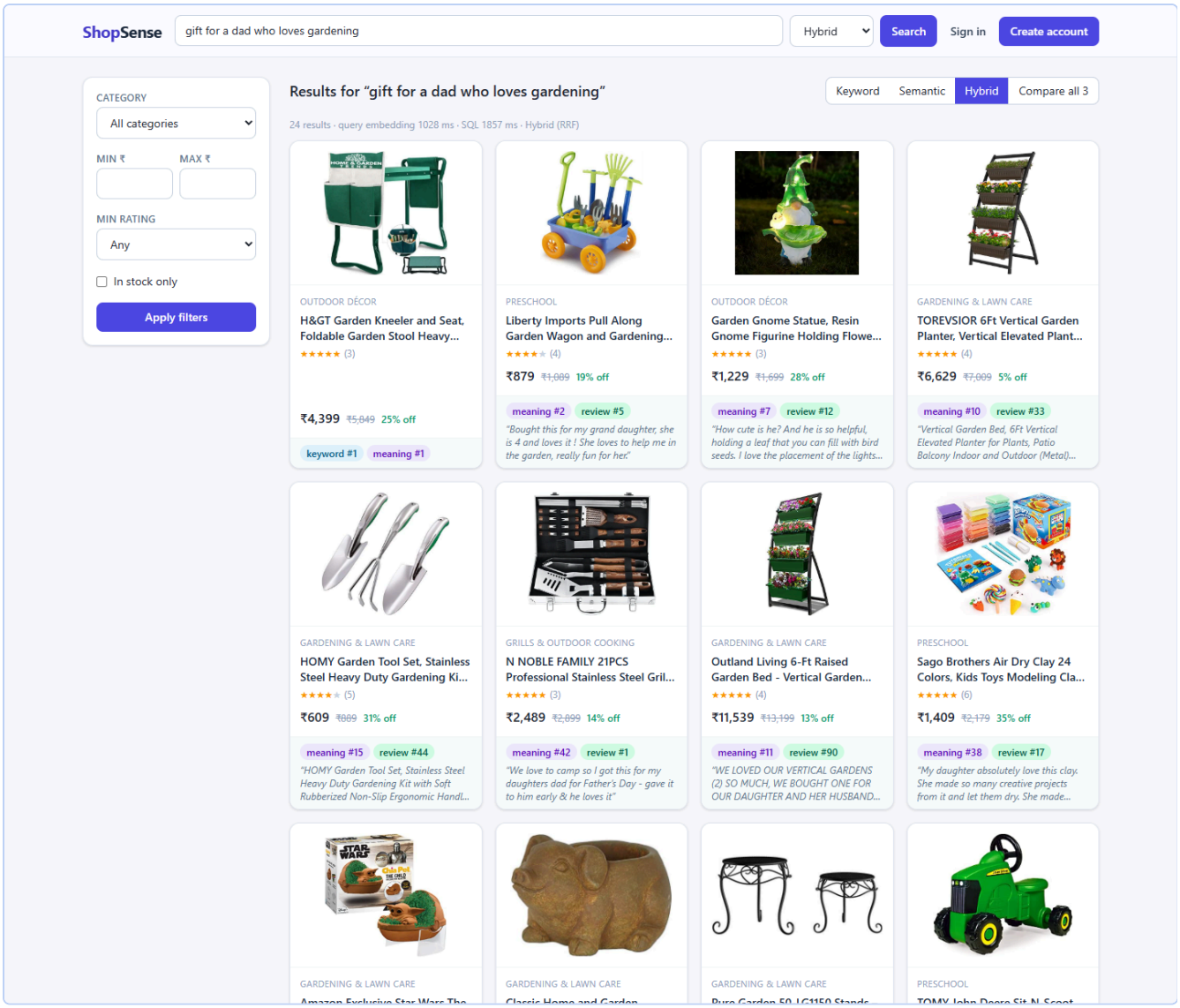
The Next.js App Router maps folders to URLs: `src/app/search/page.tsx` is `/search`, and `src/app/api/checkout/route.ts` is the `POST /api/checkout` endpoint. Pages in `src/app` are server components that call the data layer in `src/lib/data`; every database access goes through `withUser()` in `src/lib/db.ts`, which opens a transaction and sets the current user for Row-Level Security.

8. Sample Screenshots: Application Demo and Code

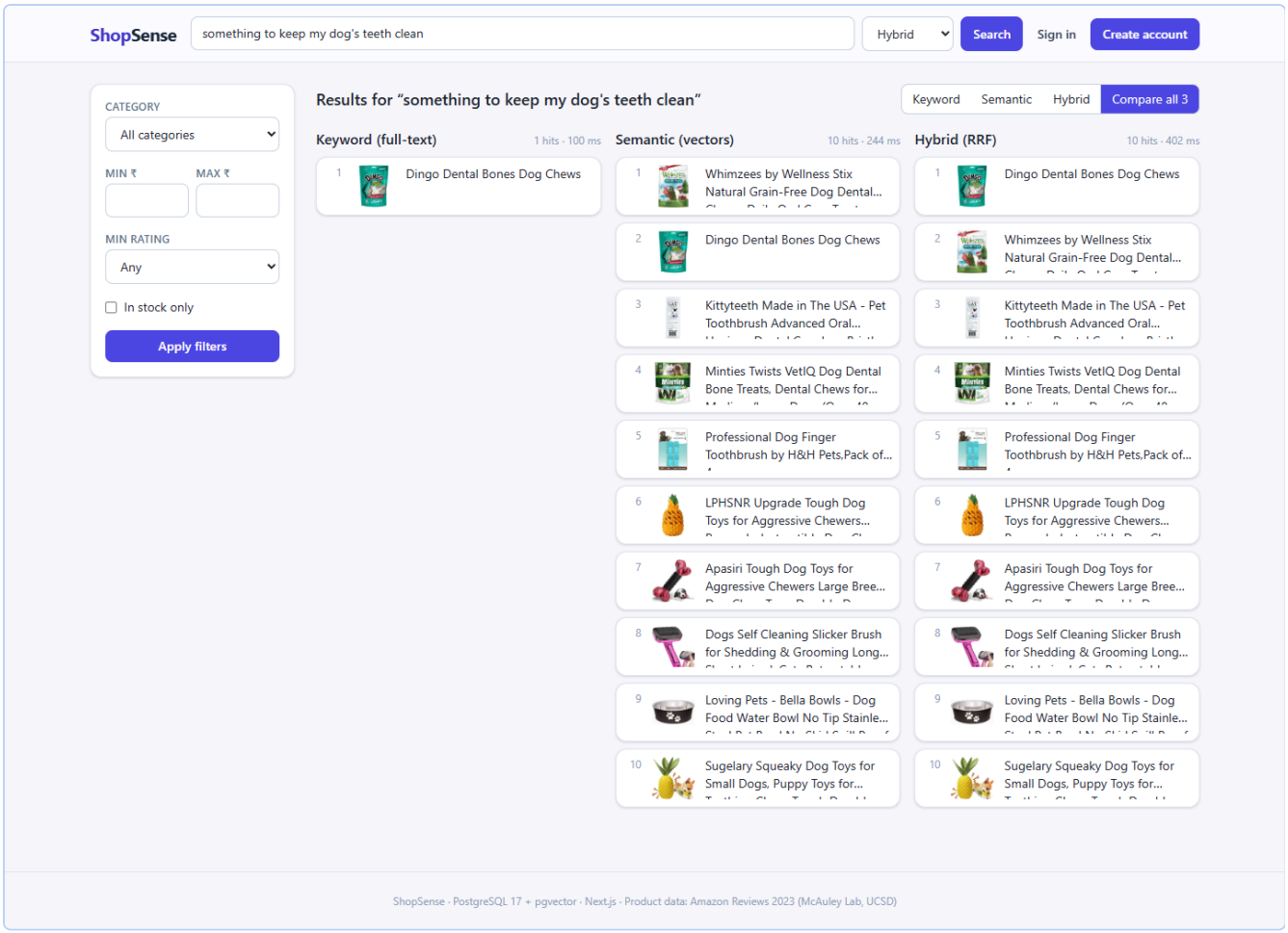
8.1 Application demo



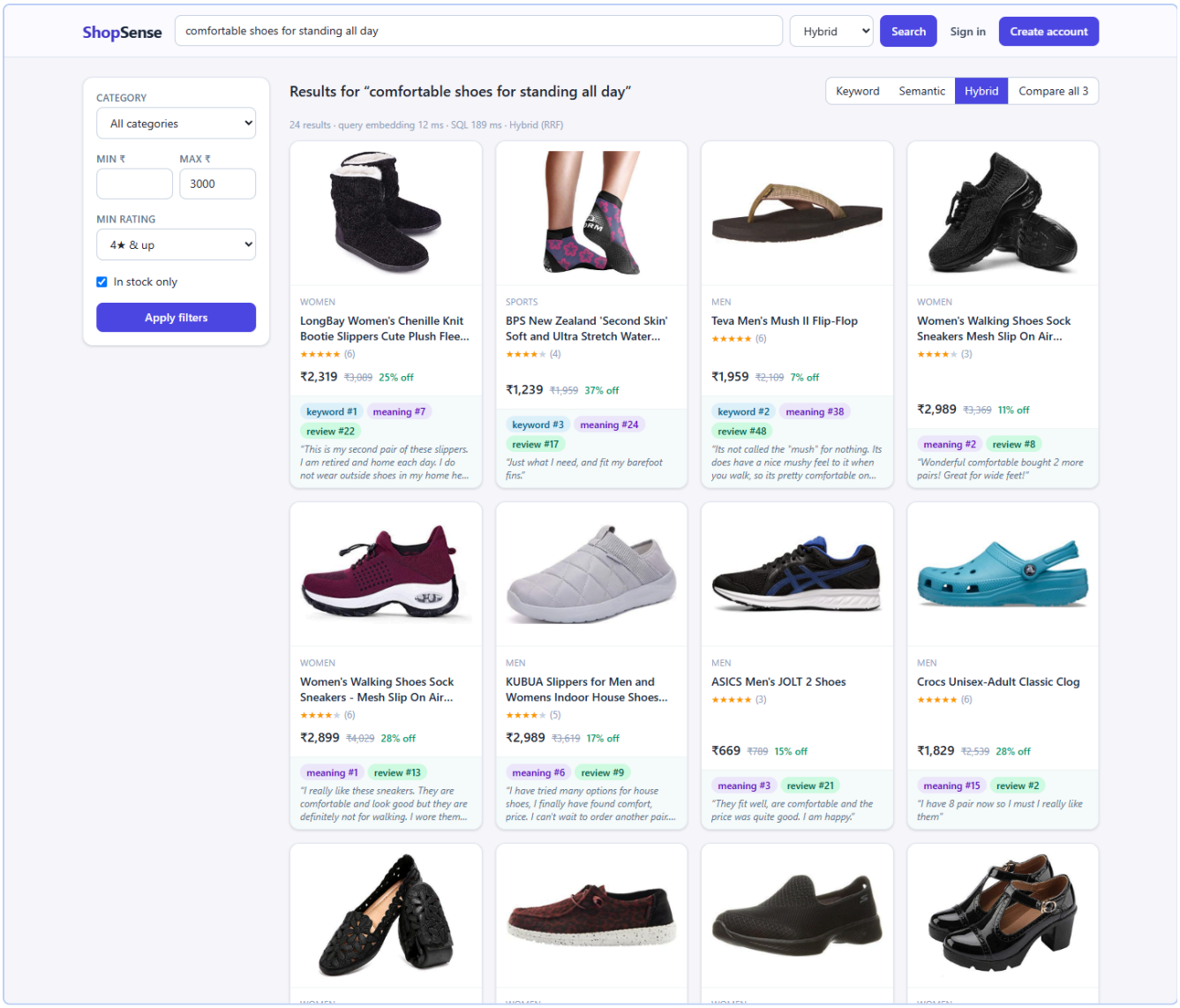
Home page: natural-language search box, example queries, categories and trending products.



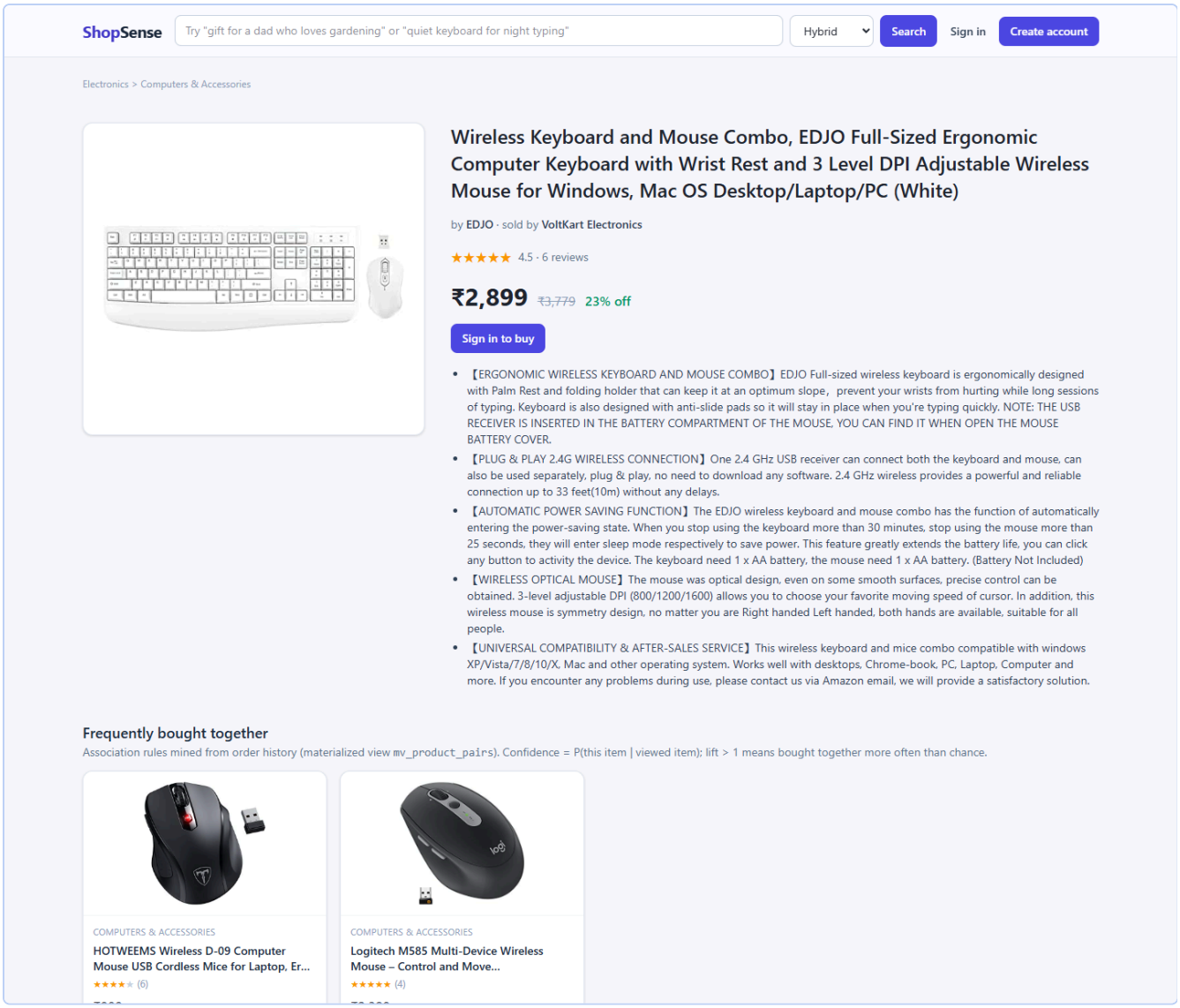
Hybrid search for "gift for a dad who loves gardening", with keyword, meaning and review ranks and the matching review quote.



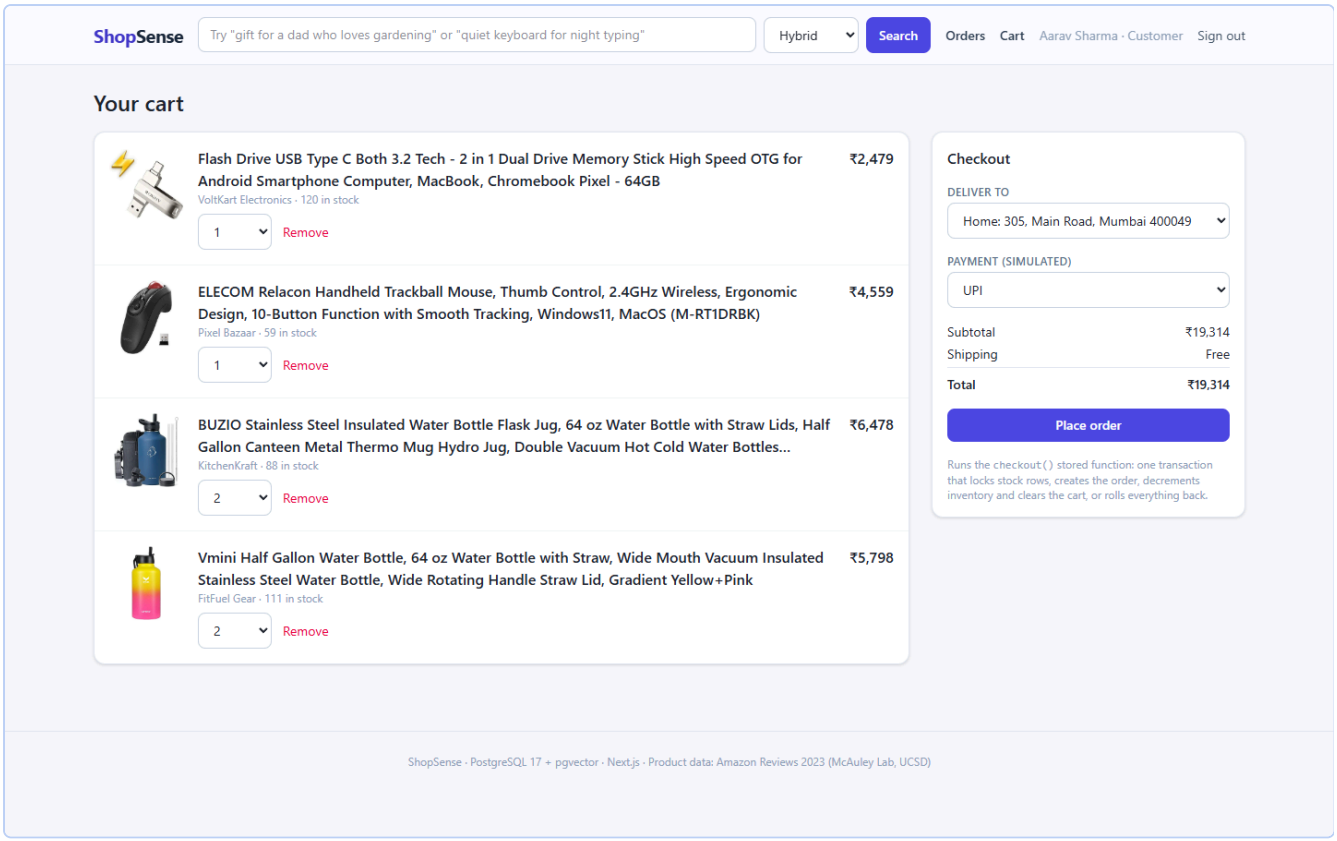
"Compare all 3": keyword, semantic and hybrid results side by side for the same query.



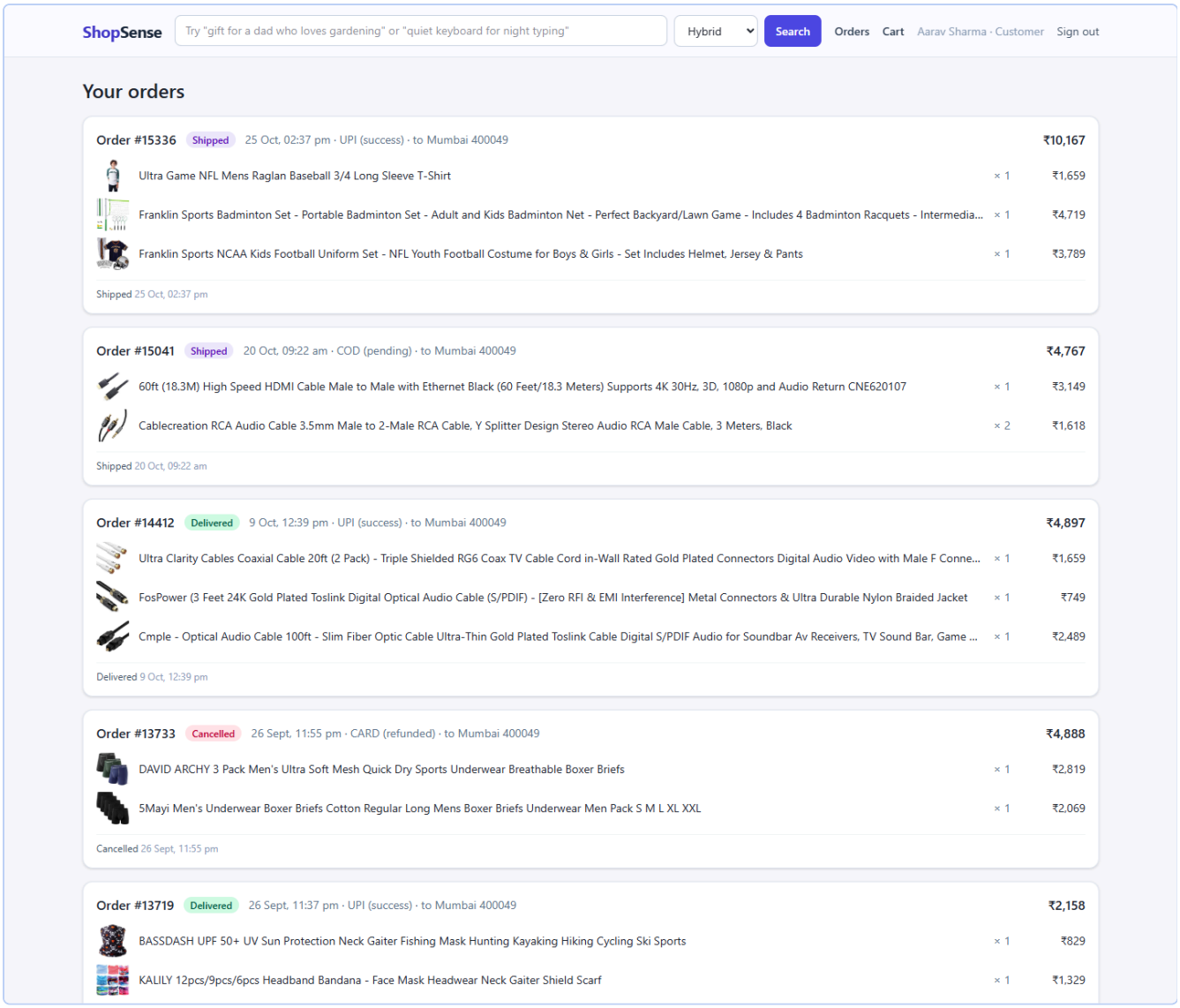
Filtered search: price, rating and stock filters are applied inside the same SQL function.



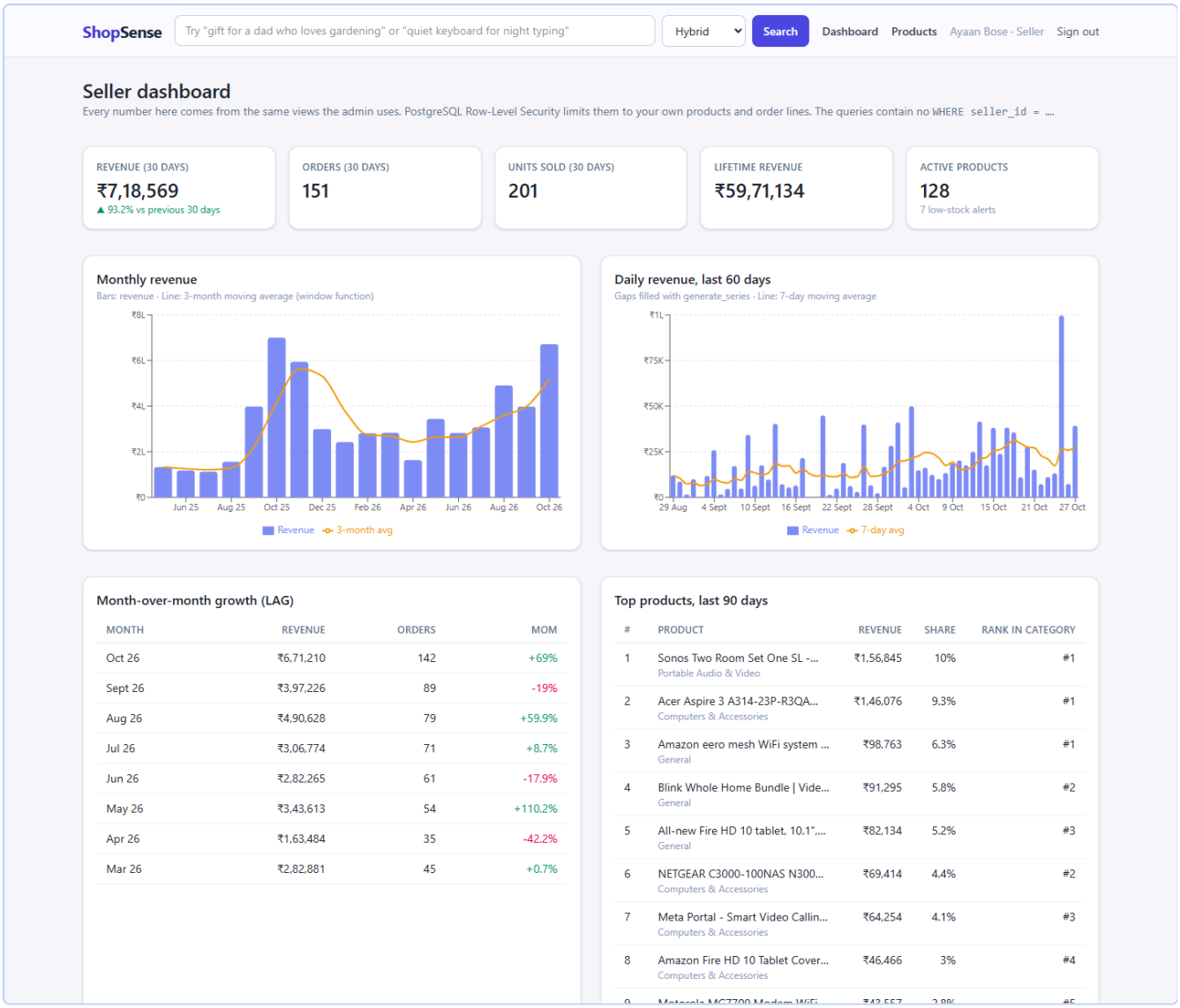
Product page: ratings, reviews with verified-purchase badges, similar products and frequently bought together.



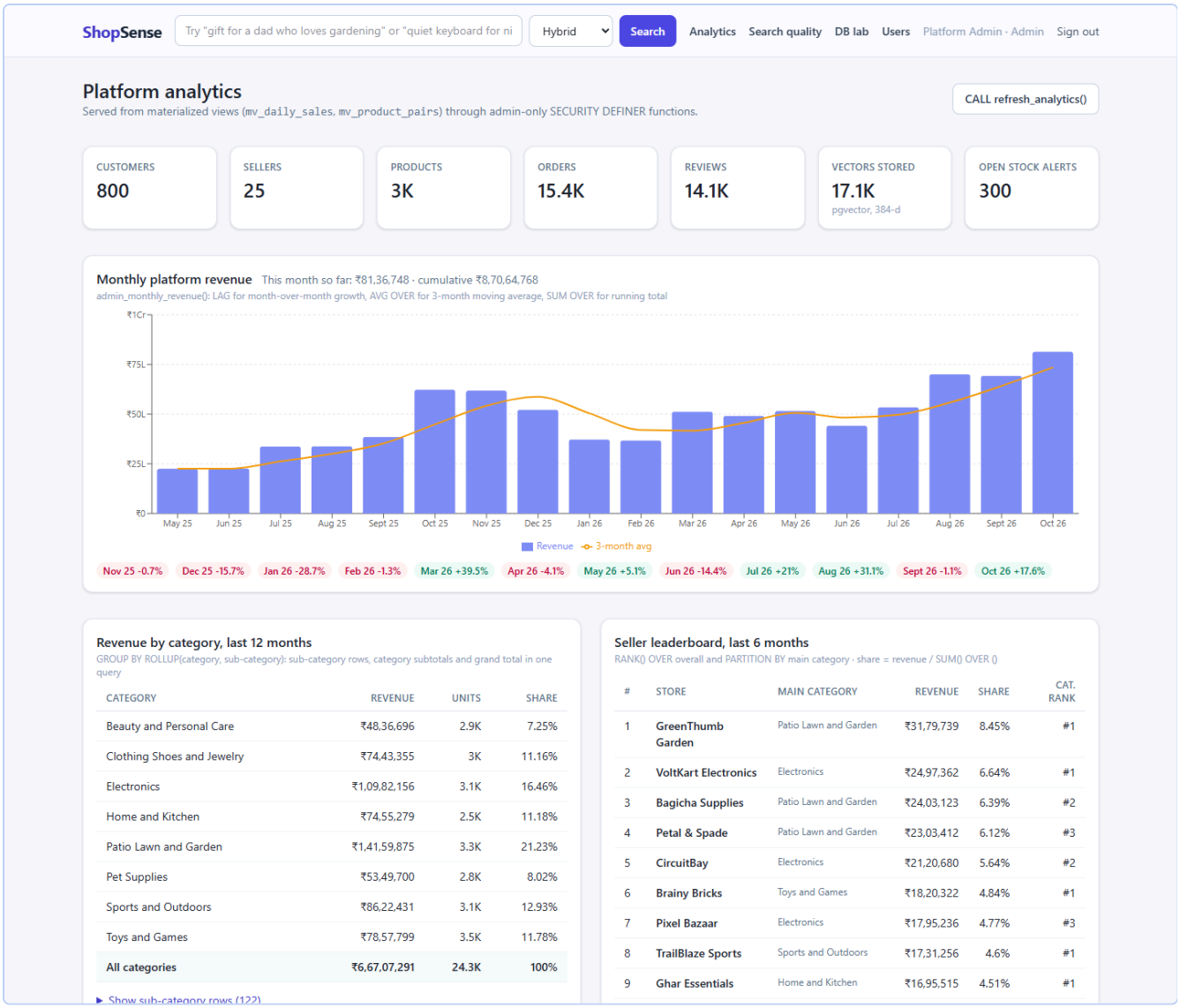
Customer cart and checkout (one atomic stored function).



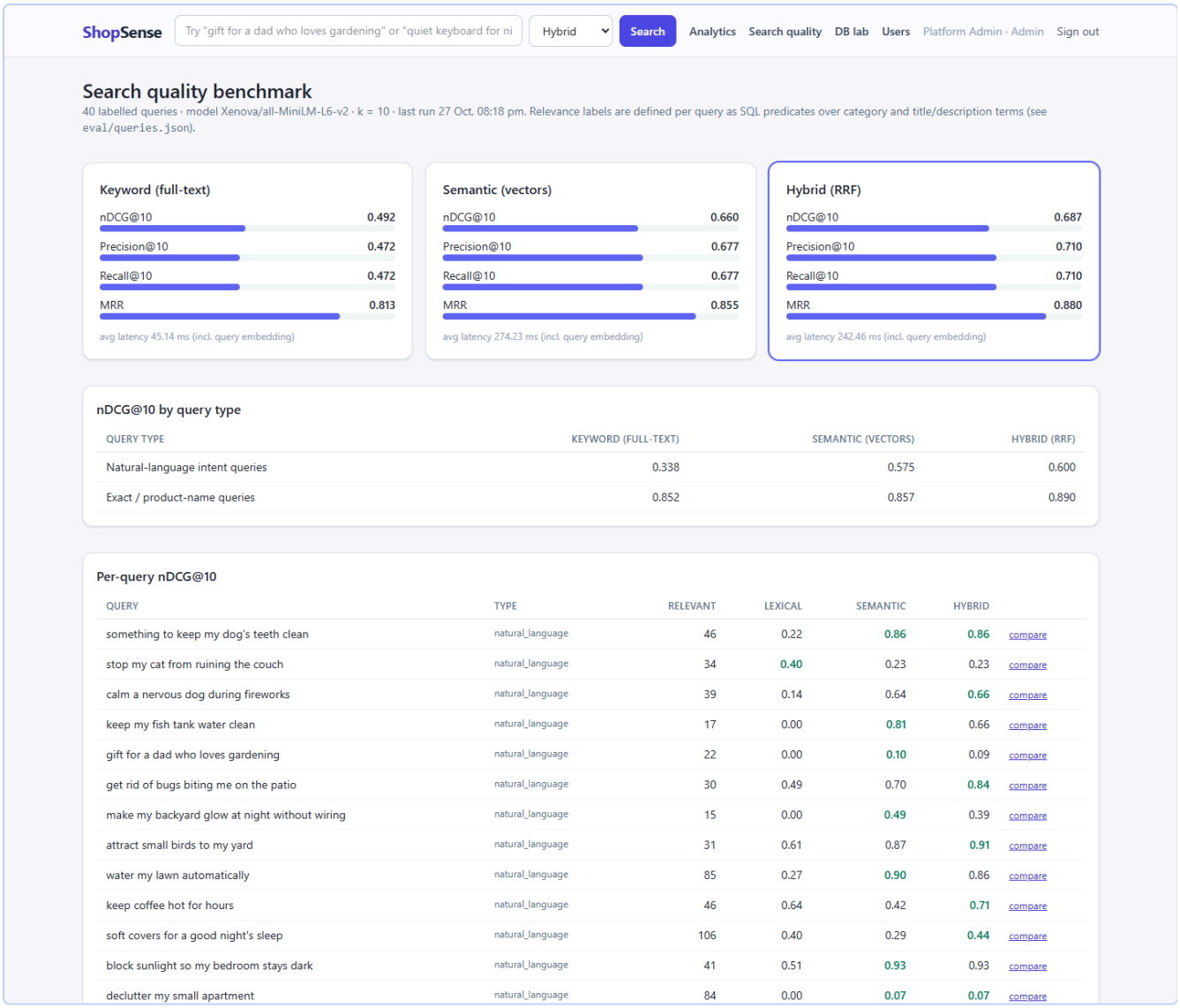
Order history with the status timeline written by the audit trigger.



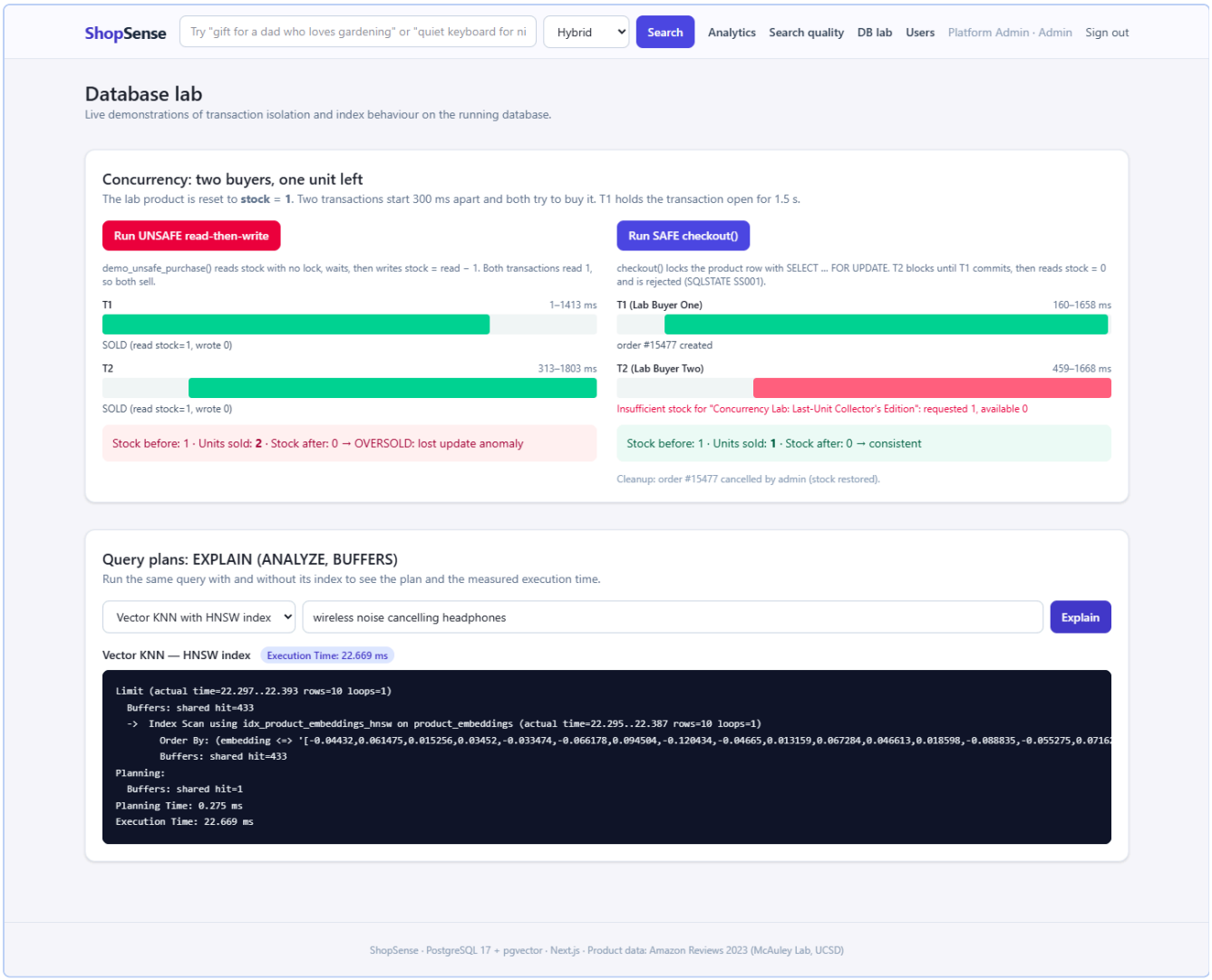
Seller dashboard: revenue trend, month-over-month growth, top products, low-stock alerts and fulfilment. All numbers are limited to the seller by Row-Level Security.



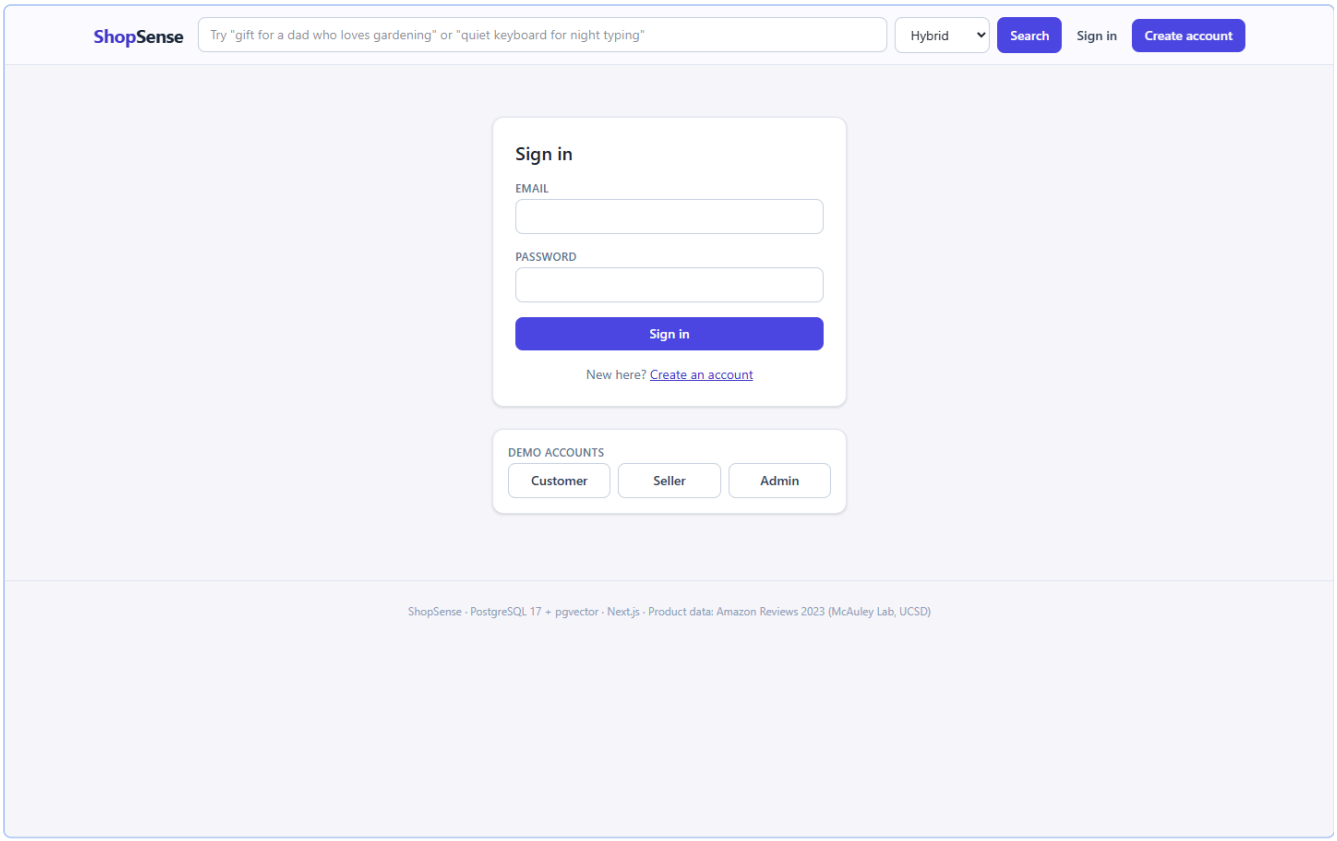
Admin analytics: category roll-up, seller leaderboard, search analytics and the embedding queue.



Search quality benchmark page with per-query results.



Database lab: the unsafe read-then-write sells the last unit twice; checkout() with a row lock rejects the second buyer. Below, an EXPLAIN ANALYZE of the HNSW index.



Login page with one-click demo accounts for each role.

8.2 Application code

db/02_schema.sql · lines 71–108

```sql
CREATE TABLE products (
  product_id         BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  seller_id          BIGINT    NOT NULL REFERENCES sellers (seller_id) ON DELETE RESTRICT,
  category_id        INT       NOT NULL REFERENCES categories (category_id) ON DELETE RESTRICT,
  asin               TEXT      UNIQUE,                          -- source identifier (dataset)
  title              TEXT      NOT NULL CHECK (length(btrim(title)) BETWEEN 3 AND 300),
  brand              TEXT,
  description        TEXT      NOT NULL DEFAULT '',
  features           TEXT      NOT NULL DEFAULT '',             -- bullet points, newline separated
  price              money_inr NOT NULL CHECK (price > 0),
  mrp                money_inr NOT NULL,
  stock_qty          INT       NOT NULL DEFAULT 0  CHECK (stock_qty >= 0),
  reorder_threshold  INT       NOT NULL DEFAULT 10 CHECK (reorder_threshold BETWEEN 0 AND 10000),
  image_url          TEXT      CHECK (image_url ~ '^https?://'),
  -- Derived attributes, maintained by trigger (trg_reviews_stats_*) for read performance
  avg_rating         NUMERIC(2, 1) NOT NULL DEFAULT 0 CHECK (avg_rating BETWEEN 0 AND 5),
  review_count       INT       NOT NULL DEFAULT 0 CHECK (review_count >= 0),
  is_active          BOOLEAN   NOT NULL DEFAULT TRUE,
  created_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  updated_at         TIMESTAMPTZ NOT NULL DEFAULT now(),
  -- Full-text search document, weighted: title (A) > brand (B) > features (C) > description (D)
  search_tsv         tsvector GENERATED ALWAYS AS (
                       setweight(to_tsvector('english', coalesce(title, '')), 'A') ||
                       setweight(to_tsvector('english', coalesce(brand, '')), 'B') ||
                       setweight(to_tsvector('english', coalesce(features, '')), 'C') ||
                       setweight(to_tsvector('english', coalesce(description, '')), 'D')
                     ) STORED,
  CONSTRAINT chk_products_mrp_ge_price CHECK (mrp >= price)
);

CREATE TABLE reviews (
  review_id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  product_id         BIGINT   NOT NULL REFERENCES products (product_id) ON DELETE CASCADE,
  user_id            BIGINT   REFERENCES users (user_id) ON DELETE SET NULL,   -- review survives account deletion
  rating             SMALLINT NOT NULL CHECK (rating BETWEEN 1 AND 5),
  title              TEXT     CHECK (length(title) <= 200),
  body               TEXT     NOT NULL CHECK (length(btrim(body)) BETWEEN 10 AND 5000),
  verified_purchase  BOOLEAN  NOT NULL DEFAULT FALSE,              -- set by trigger, not by client
```

Table definition with constraints and generated columns (db/02_schema.sql).

db/03_indexes.sql · lines 31–42

```sql
-- Full-text search: GIN over the generated tsvector
CREATE INDEX idx_products_search_tsv ON products USING GIN (search_tsv);
-- Trigram index for typo-tolerant autocomplete on titles
CREATE INDEX idx_products_title_trgm ON products USING GIN (title gin_trgm_ops);

-- Vector similarity: HNSW graph indexes using cosine distance (<=>)
-- m = max connections per node, ef_construction = candidate list size at build time
CREATE INDEX idx_product_embeddings_hnsw ON product_embeddings
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);
CREATE INDEX idx_review_embeddings_hnsw ON review_embeddings
  USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64);

```

Full-text (GIN) and vector (HNSW) indexes (db/03_indexes.sql).

db/04_functions.sql · lines 157–214

```
157 CREATE FUNCTION checkout(p_address_id BIGINT, p_method payment_method)
158 RETURNS BIGINT
159 LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
160 DECLARE
161   v_uid       BIGINT := app_user_id();
162   v_cart_id   BIGINT;
163   v_order_id  BIGINT;
164   v_subtotal  NUMERIC(12, 2) := 0;
165   v_fee       NUMERIC(12, 2);
166   v_hold      NUMERIC := coalesce(NULLIF(current_setting('app.demo_hold_seconds', true), '')::NUMERIC, 0);
167   r           RECORD;
168 BEGIN
169   IF v_uid IS NULL THEN
170     RAISE EXCEPTION 'Not authenticated' USING ERRCODE = '28000';
171   END IF;
172   IF NOT EXISTS (SELECT 1 FROM addresses WHERE address_id = p_address_id AND user_id = v_uid) THEN
173     RAISE EXCEPTION 'Address % does not belong to the current user', p_address_id USING ERRCODE = '42501';
174   END IF;
175
176   SELECT cart_id INTO v_cart_id FROM carts WHERE user_id = v_uid;
177   IF v_cart_id IS NULL OR NOT EXISTS (SELECT 1 FROM cart_items WHERE cart_id = v_cart_id) THEN
178     RAISE EXCEPTION 'Cart is empty' USING ERRCODE = 'SS002';
179   END IF;
180
181   FOR r IN
182     SELECT p.product_id, p.title, p.price, p.stock_qty, p.is_active, ci.quantity
183       FROM cart_items ci
184       JOIN products p ON p.product_id = ci.product_id
185      WHERE ci.cart_id = v_cart_id
186      ORDER BY p.product_id
187        FOR UPDATE OF p
188   LOOP
189     IF NOT r.is_active THEN
190       RAISE EXCEPTION '"%" is no longer available', r.title USING ERRCODE = 'SS003';
191     END IF;
192     IF r.stock_qty < r.quantity THEN
193       RAISE EXCEPTION 'Insufficient stock for "%": requested %, available %', r.title, r.quantity, r.stock_qty
194         USING ERRCODE = 'SS001';
195     END IF;
196     v_subtotal := v_subtotal + r.price * r.quantity;
197   END LOOP;
198
199   -- Concurrency-Lab hook: hold the row locks for a moment so a competing checkout visibly waits
200   IF v_hold > 0 THEN
201     PERFORM pg_sleep(least(v_hold, 5));
202   END IF;
203
204   v_fee := CASE WHEN v_subtotal >= 499 THEN 0 ELSE 40 END;
205
206   INSERT INTO orders (user_id, address_id, status, subtotal, shipping_fee, total_amount)
207   VALUES (v_uid, p_address_id,
208           CASE WHEN p_method = 'cod' THEN 'pending' ELSE 'paid' END::order_status,
209           v_subtotal, v_fee, v_subtotal + v_fee)
210   RETURNING order_id INTO v_order_id;
211
212   INSERT INTO order_items (order_id, product_id, quantity, unit_price)
213   SELECT v_order_id, p.product_id, ci.quantity, p.price
214     FROM cart_items ci JOIN products p USING (product_id)
```

The checkout() stored function: row locks, stock checks, order creation (db/04_functions.sql).

db/04_functions.sql · lines 356–407

```
356  and_match AS MATERIALIZED (
357    SELECT p.product_id
358      FROM products p, q
359     WHERE p_mode IN ('lexical', 'hybrid')
360       AND q.tsq_and::TEXT <> ''
361       AND p.search_tsv @@ q.tsq_and
362  ),
363  and_hits AS (
364    -- does any eligible product contain every query term?
365    SELECT count(*) AS n FROM and_match WHERE product_id IN (SELECT product_id FROM eligible)
366  ),
367  lex_q AS (
368    SELECT CASE WHEN a.n > 0 THEN q.tsq_and ELSE q.tsq_or END AS tsq,
369           CASE WHEN a.n > 0 THEN 1.0 ELSE 0.25 END           AS weight
370      FROM q, and_hits a
371  ),
372  lex_match AS MATERIALIZED (
373    SELECT p.product_id, ts_rank_cd(p.search_tsv, lq.tsq, 1) AS rank
374      FROM products p, lex_q lq
375     WHERE p_mode IN ('lexical', 'hybrid')
376       AND lq.tsq IS NOT NULL
377       AND p.search_tsv @@ lq.tsq
378  ),
379  lexical AS (
380    SELECT x.product_id, row_number() OVER (ORDER BY x.rank DESC, x.product_id)::INT AS rnk
381      FROM (SELECT m.product_id, m.rank
382              FROM lex_match m
383             WHERE m.product_id IN (SELECT product_id FROM eligible)
384             ORDER BY m.rank DESC
385             LIMIT 100) x
386  ),
387  semantic AS (
388    SELECT x.product_id, x.dist, row_number() OVER (ORDER BY x.dist, x.product_id)::INT AS rnk
389      FROM (SELECT pe.product_id, pe.embedding <=> p_embedding AS dist
390              FROM product_embeddings pe
391             WHERE p_mode IN ('semantic', 'hybrid')
392               AND pe.product_id IN (SELECT product_id FROM eligible)
393             ORDER BY pe.embedding <=> p_embedding
394             LIMIT 100) x
395  ),
396  review_hits AS (
397    SELECT r.product_id, re.review_id, re.embedding <=> p_embedding AS dist
398      FROM review_embeddings re
399      JOIN reviews r USING (review_id)
400     WHERE p_mode IN ('semantic', 'hybrid')
401       AND r.product_id IN (SELECT product_id FROM eligible)
402     ORDER BY re.embedding <=> p_embedding
403     LIMIT 300
404  ),
405  review_best AS (
406    SELECT DISTINCT ON (h.product_id) h.product_id, h.review_id, h.dist
407      FROM review_hits h
```

Hybrid search: keyword match with fallback, then Reciprocal Rank Fusion (db/04_functions.sql).

db/05_triggers.sql · lines 43–72

```sql
CREATE FUNCTION trg_fn_stock_alert() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
BEGIN
  IF NEW.stock_qty < NEW.reorder_threshold THEN
    INSERT INTO stock_alerts (product_id, stock_qty, threshold)
    VALUES (NEW.product_id, NEW.stock_qty, NEW.reorder_threshold)
    ON CONFLICT (product_id) WHERE resolved_at IS NULL
    DO UPDATE SET stock_qty = EXCLUDED.stock_qty, threshold = EXCLUDED.threshold;
    PERFORM pg_notify('stock_alerts', json_build_object(
      'product_id', NEW.product_id, 'seller_id', NEW.seller_id, 'stock_qty', NEW.stock_qty)::TEXT);
  ELSE
    UPDATE stock_alerts SET resolved_at = now()
     WHERE product_id = NEW.product_id AND resolved_at IS NULL;
  END IF;
  RETURN NULL;   -- AFTER trigger: return value ignored
END $$;

CREATE TRIGGER trg_products_stock_alert
  AFTER INSERT OR UPDATE OF stock_qty, reorder_threshold ON products
  FOR EACH ROW EXECUTE FUNCTION trg_fn_stock_alert();

-- ---------------------------------------------------------------------
-- T4. EMBEDDING CONSISTENCY
-- Whenever text that feeds an embedding changes, queue a re-embedding job and NOTIFY the worker.
-- ON CONFLICT against the partial unique index de-duplicates bursts of edits.
-- ---------------------------------------------------------------------
CREATE FUNCTION trg_fn_enqueue_embedding() RETURNS TRIGGER
LANGUAGE plpgsql SECURITY DEFINER SET search_path = public AS $$
DECLARE
  v_type TEXT := TG_ARGV[0];
```

Low-stock alert trigger (db/05_triggers.sql).

db/07_security.sql · lines 122–143

```sql
-- orders: customer sees own; seller sees orders that contain their products; admin sees all
CREATE POLICY orders_select ON orders FOR SELECT
  USING (user_id = (SELECT app_user_id())
         OR (SELECT is_admin())
         OR ((SELECT app_role()) = 'seller' AND order_id IN (SELECT my_seller_order_ids())));
-- status updates (ship / deliver) by admin or a seller involved in the order;
-- the state-machine trigger still decides which transitions are legal
CREATE POLICY orders_update ON orders FOR UPDATE
  USING ((SELECT is_admin()) OR ((SELECT app_role()) = 'seller' AND order_id IN (SELECT my_seller_order_ids())));

-- order lines: a seller sees ONLY the lines for their own products, even inside a shared order
CREATE POLICY order_items_select ON order_items FOR SELECT
  USING (order_id IN (SELECT my_order_ids())
         OR (SELECT is_admin())
         OR product_id IN (SELECT product_id FROM products WHERE seller_id = (SELECT app_user_id())));

CREATE POLICY payments_select ON payments FOR SELECT
  USING (order_id IN (SELECT my_order_ids()) OR (SELECT is_admin()));

CREATE POLICY status_history_select ON order_status_history FOR SELECT
  USING (order_id IN (SELECT my_order_ids())
         OR (SELECT is_admin())
```

Row-Level Security policies for orders (db/07_security.sql).

src/lib/db.ts · lines 28–45

```ts
export async function withUser<T>(session: Session | null, fn: (c: PoolClient) => Promise<T>): Promise<T> {
  const client = await pool.connect();
  try {
    await client.query('BEGIN');
    await client.query(`SELECT set_config('app.user_id', $1, true)`, [session ? String(session.uid) : '']);
    const result = await fn(client);
    await client.query('COMMIT');
    return result;
  } catch (err) {
    await client.query('ROLLBACK').catch(() => {});
    throw err;
  } finally {
    client.release();
  }
}

/** Anonymous read (no user context). RLS still applies. */
export function publicQuery<T>(fn: (c: PoolClient) => Promise<T>) {
```

Transaction helper that sets the current user for Row-Level Security (src/lib/db.ts).

src/app/api/checkout/route.ts · lines 1–21

```ts
import { NextResponse } from 'next/server';
import { z } from 'zod';
import { withUser } from '@/lib/db';
import { handler, requireRole } from '@/lib/auth';

const Body = z.object({
  addressId: z.number().int().positive(),
  method: z.enum(['upi', 'card', 'netbanking', 'cod']),
});

// The whole checkout (stock locks, order, items, stock decrement, payment, cart clear)
// is one atomic stored function: checkout(). Errors such as insufficient stock (SQLSTATE SS001)
// roll the transaction back and surface as HTTP 409.
export const POST = handler(async (req: Request) => {
  const s = await requireRole('customer');
  const { addressId, method } = Body.parse(await req.json());
  const orderId = await withUser(s, async (c) =>
    (await c.query('SELECT checkout($1, $2::payment_method) AS order_id', [addressId, method])).rows[0].order_id);
  return NextResponse.json({ orderId }, { status: 201 });
});

```

The checkout API route handler (src/app/api/checkout/route.ts).

8.3 Testing

`npm test` runs 22 integration tests against the live database. They connect as the least-privilege application role, create their own fixtures and remove them afterwards. **Result: 22 passed, 0 failed.**

Area	What the tests check
Authentication	correct and wrong passwords, unknown user, case-insensitive email, bcrypt storage, password hash unreadable by the app role
Validation	weak password, duplicate email, bad email, phone, PIN code and GSTIN, price and stock limits, rating range, short review, non-seller cannot become a seller
Checkout	order, price snapshot, free shipping threshold, stock decrement, cart cleared, payment and audit rows; empty cart; over-stock; someone else's address
Concurrency	two parallel checkouts of the last unit: exactly one succeeds

Area	What the tests check
Orders	cancellation restores stock and refunds; only the owner can cancel; illegal status changes are rejected; status history recorded
Triggers	low-stock alert opens, updates and resolves; verified-purchase is computed by the database; rating aggregates; embedding queue enqueues text edits but not price edits
Row-Level Security	seller cannot change another seller's product; customer, sellers and anonymous users each see only their rows; users table is private; admin-only functions reject others
Search	keyword search with filters, hybrid search ordering, category recursion, association rules

Bugs the tests and benchmarks caught, all fixed: trigger functions lacked rights to write their side-effect tables (now `SECURITY DEFINER`); a generic trigger read a column missing on one table; the first hybrid fusion design (above); and a slow keyword query plan that ignored the GIN index.

9. Evaluation Rubrics: Completion Status

#	Requirement (from the project document)	Status	Where it is
1	Unique project title; real-world problem; not simple CRUD	**Done**	Section 1, 3. Title to be confirmed as unique in the section
2	Team of 2 with defined responsibilities across database and application	**Done**	Responsibilities split by feature (below)
3	SQL (PostgreSQL) plus Vector database (pgvector)	**Done**	Sections 4, 6
4	ER diagram, relational schema, keys, constraints; vector data model documented	**Done**	`docs/ER-diagram.md`, `docs/data-dictionary.md`, `docs/vector-database-design.md`
5	DDL: CREATE, ALTER, DROP, keys, constraints, indexes, views	**Done**	Section 5.1 to 5.4, 5.7
6	DML: INSERT, UPDATE, DELETE, SELECT with JOIN, GROUP BY, HAVING, ORDER BY, aggregates, subqueries	**Done**	Section 5.5, 5.6
7	At least four advanced SQL concepts and one vector concept	**Done**	Transactions, views and materialized views, stored functions and procedures, triggers, indexing, full-text search, window functions, Row-Level Security; vector similarity search
8	Vector explanation: what is embedded, model, storage, metric, retrieval	**Done**	Section 6
9	Functional Next.js interface with forms, tables, search, filters, dashboards	**Done**	Section 8.1
10	Next.js talks to the database through an API layer; no hard-coded credentials; environment variables	**Done**	Route handlers in `src/app/api`; `.env.example`, `.env` is git-ignored
11	Backend authentication and at least two roles	**Done**	Customer, seller, admin; JWT sessions; bcrypt in the database
12	Input validation at the database level; consistency and integrity demonstrated	**Done**	Domains, CHECK constraints, triggers, transactions; tests in Section 8.3

#	Requirement (from the project document)	Status	Where it is
13	Documentation: objectives, requirements, ER, schema, DB justification, data dictionary, implementation, queries, architecture, screenshots, testing, conclusion	**Done**	This report and `docs/`
14	Git and GitHub; commits show individual contributions; README with setup	**Partly done**	Public repository and README are done. Commits so far come from one account; the teammate's own work should be committed under their name
15	Demonstration: design, schema, implementation, queries, interface, integration, advanced features, test cases, individual contribution	**Ready**	Demo script in the README; `npm test` ; `/admin/lab`
16	Originality	**Done**	Original design and implementation; uses a public dataset (credited) for product and review text
17	Level 3 expectation: beyond CRUD, database choice justified by the data	**Done**	Section 4, 6
–	Deliverable: project report	**Done**	This document
–	Deliverable: public GitHub repository link	**Done**	https://github.com/Akshatpy/DBMS_MiniProject
–	Individual contribution, modifications and viva	**To prepare**	Viva guide in `docs/`

Division of work. Member A: schema and constraints, checkout and concurrency, triggers, authentication and Row-Level Security, seller and admin dashboards. Member B: data ingestion and embedding pipeline, hybrid search, search benchmark, association rules and recommendations, product pages and the index benchmark.

Known limitations. Payments and shipping are simulated. Orders and users are synthetic; only product and review text are real. Relevance labels are pattern-based rather than human-annotated, and the benchmark is small, so absolute numbers should not be compared with published benchmarks.